เฉลยและเกณฑ์ตรวจ: CPE333 Mock Midterm ชุดที่ 1

สำหรับตรวจหลังทำข้อสอบเท่านั้น ไม่ใช่เฉลยข้อสอบจริงของรายวิชา
คะแนนรวม 50: ปรนัย $20c/41$ เมื่อ c คือจำนวนข้อถูก; เติมคำ 10; อัตนัย $8 + 4 + 8 = 20$
ไม่ปัดคะแนนรายข้อ; เมื่อสรุปผลจึงปัดคะแนนรวมเป็นทศนิยมสองตำแหน่ง ไม่มีการหักคะแนนตอบผิด

เฉลยส่วนที่ 1: คำตอบเดียวที่ดีที่สุด

คงถ้อยคำตัวเลือกจากข้อสอบเก่าตามต้นฉบับ; หมายเหตุด้านล่างแยกการตรวจแนวคิดออกจากการแก้ข้อความต้นฉบับ.

1. **(a)** This is the option highlighted in the old exam. Technically, a trap transfers control to an OS handler; a system call may invoke a trap, so the source wording equates related but distinct concepts. [L1; E09 Q1]

2. **(c)** Without user/kernel-mode separation, a process may bypass memory protection and access another process's data. [L1; E09 Q2]

3. **(e)** The Lecture 2 model is Process = (Program, Data, Control state, Process state); all four are components. [L2; E09 Q4]

4. **(b)** Isolation protects one application from unauthorized access by another. [L1; E09 Q5]

5. **(b)** A timer interrupt lets a preemptive scheduler regain control from a CPU-bound process. The source says ``non-preemptive''; that premise was corrected because a timer interrupt enables preemption. [L2; E09 Q6]

6. **(d)** The process descriptor/PCB is maintained in kernel space; code, data, and stack are user-space regions. [L2; E09 Q7]

7. **(c)** In the conventional layout used in the course, the stack grows opposite to the heap. [L2; E09 Q8]

8. **(d)** A process must be running before it can request I/O or another operation that blocks it; there is no direct Ready-to-Blocked transition. The source key's ``reads a file'' answer was corrected. [L2; E09 Q9]

9. **(d)** Process creation, I/O completion, and a timer-driven preemption can all place a process in the ready state in this model. [L2; E09 Q10]

10. **(d)** A zombie has terminated but remains represented until its parent collects its exit status. [L2; E09 Q12]

11. **(a)** The PCB stores per-process control information such as state and saved CPU context. [L2; E09 Q13]

12. **(a)** This is the option highlighted in the old exam. Read it as the parent still being alive after the child exits and before the parent collects the child's exit status; the printed choice is abbreviated and imprecise. [L2; E09 Q31]

13. **(e)** This is the answer indicated by the old lab question: the parent prints after the child finishes. A call to wait() alone does not guarantee that the child itself printed; that detail depends on the referenced lab program. [L2; E09 Q14]

14. **(d)** Two successful fork calls leave four processes to execute the print statement once each. [L2; E03 Q7]

15. **(e)** Timer preemption, blocking for I/O or wait, and yielding can each release the CPU. [L2; E09 Q16]

16. **(a)** FIFO runs each job to completion in arrival order. The source's broader wording and marked ``Both FIFO and RR'' can describe ready-queue rotation, so the stem is narrowed to completion order to keep one answer. [L3; E09 Q15]

17. **(b)** In the C storage-class model covered in class, register requests register storage for a local variable; the source's ``fastest'' claim is not guaranteed by C. [L4; E09 Q19]

18. **(c)** Fixed-size partitions can waste unused space inside an allocated partition, which is internal fragmentation. The scanned source marks this choice; a separate typed key saying ``none'' is incorrect. [L4; E09 Q20]

19. **(b)** The short-term scheduler selects a ready process to run. [L3; E09 Q21]

20. **(c)** Page replacement chooses a resident page to remove when a frame is needed. [L6; E09 Q22]

21. **(e)** Paging uses page tables, segmentation uses segment tables, and a TLB may cache translations; these mechanisms are not all required simultaneously. [L5; E09 Q23]

22. **(c)** The stem is narrowed to ask specifically for the fragmentation problem; the source choices are reproduced verbatim. The old key marked (d), which names a separate claimed cost rather than a fragmentation type. [L5; E09 Q24]

23. **(a)** Demand paging loads a page when it is referenced and is not already resident. [L6; E09 Q26]

24. **(e)** Context saving, victim selection when needed, page-in, and page-table update can all occur while handling a valid page fault. [L6; E09 Q27]

25. **(a)** The stem is narrowed to ask for the starting physical address field, so *Base* is the single answer. The source's broad stem and marked ``All choices'' option are not consistent with the base-and-bound entry described in Lecture 5; the printed choices remain unchanged. [L5; E09 Q29]

26. **(c)** With a fixed virtual address space, a larger page means fewer virtual pages and therefore a smaller page table. [L5; E09 Q30]

27. **(c)** Round robin is considered fair because it shares CPU time among active jobs on a short time scale. [L3; E09 Q33]

28. **(a)** FIFO does not use time-slice preemption; it runs the current job until completion or blocking. [L3; E09 Q34]

29. **(e)** This is the answer marked in the old exam: the memory image is kept on secondary storage. The original distractor ``virtual storage'' is retained as printed. [L6; E09 Q35]

30. **(c)** Response time is the interval from arrival until the process first gets the CPU. [L3; E09 Q36]

31. **(b)** Turnaround time is completion time minus arrival time. [L3; E09 Q37]

32. **(d)** Compaction may be needed to address external fragmentation from variable-size partitions. The source labels compaction as the problem; the stem now asks for the operation that addresses it. [L4; E09 Q38]

33. **(a)** A larger virtual address space requires more page-table entries when page and PTE sizes are fixed. [L5; E09 Q39]

34. **(e)** A multilevel page table can omit lower-level tables for unused regions; this is how it saves memory for sparse address spaces. [L5; E09 Q40]

35. **(a)** In the simplified state model used here, a newly created process is placed in the ready state. [L2; E09 Q41]

36. **(b)** A process using the processor is in the running state. [L2; E09 Q42]

37. **(e)** A long job can starve under SJF if shorter jobs keep arriving. [L3; E09 Q44]

38. **(d)** Round robin improves response time by giving each ready job a time slice promptly. [L3; E09 Q46]

39. **(d)** Pages are units of logical memory; page frames are equal-sized units of physical memory. [L5; E09 Q48]

40. **(a)** OPT is an ideal benchmark because it requires knowledge of future page references. [L6; E09 Q49]

41. **(a)** This is the option highlighted in the old exam. Read ``not valid in page table'' as a valid virtual page whose entry indicates it is not resident; strictly, an illegal virtual address is not the same as a not-present page and may cause a protection fault instead. [L6; E09 Q50]

เฉลยส่วนที่ 2: เติมคำ / ตอบสั้น

ข้อละ 1 คะแนน ให้ partial credit ตามองค์ประกอบในแต่ละข้อ; รับถ้อยคำอื่นที่มีความหมายถูกต้อง ไม่บังคับจำประโยคเฉลย

1. **MLFQ: Multilevel Feedback Queue. STCF: Shortest Time-to-Completion First.**
   0.5 for each correct full name. Accept Multi-level and equivalent hyphenation.; สองคำย่อที่ใช้จริงใน Lecture 3 และโจทย์ scheduling ไม่ใช่คำย่ออุปกรณ์พื้นฐานเรียงหลายข้อ

2. **A program is passive code; a process is a program in execution.**
   0.5 for passive code; 0.5 for a program in execution. No resource list or additional explanation required.; Process ไม่ใช่แค่ไฟล์ program ใน disk; program เดียวมีหลาย process ได้

3. **Memory Management Unit; translates virtual addresses to physical addresses.**
   0.5 for the full name; 0.5 for virtual-to-physical translation. No protection details required.; แทนข้อ cooperative ที่ซ้ำปรนัย เพื่อเพิ่มคำย่อและความเข้าใจกลไกหน่วยความจำ

4. **Program counter (PC) and stack pointer (SP).**
   0.5 each for two distinct valid items, such as PC, SP, a general-purpose register, or the processor status register. No switching steps required.; ปรับจากแนว context switch ให้ถามเพียงตัวอย่าง 2 อย่าง ไม่บังคับอธิบายขั้นตอนเพิ่ม

5. **Process ID (PID) and process state.**
   0.5 each for two distinct non-register information types, such as PID, state, scheduling information, or memory-management information. No PCB expansion required.; ถามข้อมูล 2 ประเภท ไม่ถามชื่อเต็มพ่วงอีกส่วน และแยกจากตัวอย่าง register ในข้อก่อนหน้า

เฉลยส่วนที่ 2: Short Answer (ต่อ)

6. **Zombie process.**
   1 point for zombie or zombie process. No reason required; do not substitute orphan.; ข้อนี้ถาม keyword เท่านั้น ไม่หักคะแนนเพราะไม่ได้อธิบายเหตุผลที่โจทย์ไม่ได้ถาม

7. **I/O-intensive: copying a large file; CPU-intensive: matrix multiplication.**
   0.5 for each correctly labelled task. Accept equivalent examples; no definitions or burst analysis required.; ปรับถ้อยคำให้ตรงกับแนว workload ใน Lecture 3 มากกว่าคำว่า I/O-bound ซึ่งไม่ได้ใช้เป็นคำหลักในสไลด์

8. **Thrashing occurs when a process is busy swapping pages in and out.**
   1 point for describing repeated page swapping between memory and disk; a high page-fault rate alone is not the definition. No working-set calculation required.; ใช้นิยามตรงจาก Lecture 6 แทน LJF ซึ่งไม่อยู่ในเนื้อหา Lecture 1–6 โดยตรง

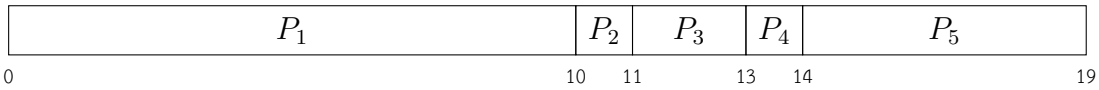
9. **No; 4200 exceeds the 4096-byte bound.**
   0.5 for No; 0.5 for the correct bounds comparison. No physical-address calculation or fault-handling steps required.; ต้องตรวจ bounds ก่อนบวก base; คำตอบ 20584 เป็นการคำนวณโดยข้าม protection check

10. **Translation Lookaside Buffer; the translation may be uncached while the page is still in RAM.**
    0.5 for the full name; 0.5 for cached-translation miss versus nonresident-page distinction.; TLB เป็น cache ของ translations ไม่ใช่ page contents จึง miss ได้ทั้งที่ page ยังอยู่ RAM

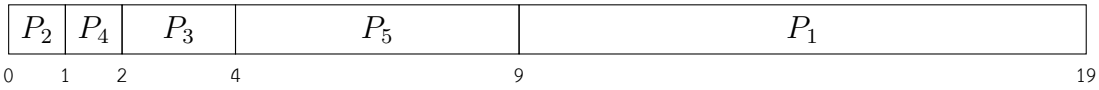
เฉลยอัตนัยข้อ 1: CPU Scheduling (8 คะแนน)

Burst = (10,1,2,1,5) ms; arrival = 0 ทุกงาน; quantum = 1 ms; เฉลยใช้ overhead 0 และให้ SJF ตัดสินงานยาวเท่ากันตามลำดับมาถึง เป็นข้อตกลงในการทำเฉลย ไม่ใช่ข้อความที่เพิ่มในโจทย์ รับลำดับ tie อื่นที่สอดคล้องกับ SJF และคำนวณถูกต้อง

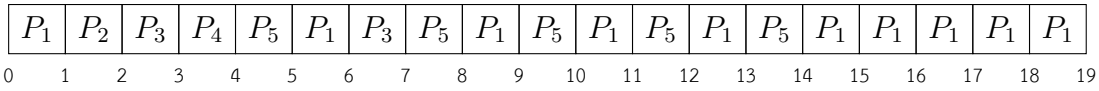
FIFO



SJF



RR



RR ยังคงมีจุดแบ่ง quantum แม้เหลือเพียง P_1; จะรวมช่วงติดกันของ P_1 ตอนท้ายก็ได้ถ้าระบุ quantum ถูกต้อง

นิยามที่ใช้

$T_i = C_i - A_i$; $R_i = S_i - A_i$; $W_i = T_i - B_i$ (กรณีมี CPU burst เดียว ไม่มี I/O)

C: completion, A: arrival, S: first start, B: CPU burst

Response time ไม่ใช่ total waiting time โดยเฉพาะเมื่อใช้ RR

เฉลยอัตนัยข้อ 1: ไล่ FIFO และ SJF

FIFO: ทุก process มาถึงเวลาเดียวกัน ลำดับตามโจทย์จึงเป็น P_1, P_2, P_3, P_4, P_5. FIFO ทำงานจนจบทีละตัว ดังนั้นเวลาสิ้นสุดได้จาก burst สะสม:

$$0 + 10 = 10, \qquad 10 + 1 = 11, \qquad 11 + 2 = 13, \qquad 13 + 1 = 14, \qquad 14 + 5 = 19.$$

Process	Burst (ms)	CPU interval	First start S_i	Completion C_i
P_1	10	$[0, 10)$	0	10
P_2	1	$[10, 11)$	10	11
P_3	2	$[11, 13)$	11	13
P_4	1	$[13, 14)$	13	14
P_5	5	$[14, 19)$	14	19

เช่น P_3 เริ่มหลัง P_1 และ P_2 จบที่ $10 + 1 = 11$; ทำงาน 2 ms จึงเสร็จที่ $11 + 2 = 13$.

SJF แบบ ไม่ แทรก งาน: เนื่องจาก ทุก งาน พร้อม ตั้งแต่ $t = 0$, เลือก burst สั้น ที่สุด ก่อน. เรียง ได้ $P_2(1), P_4(1), P_3(2), P_5(5), P_1(10)$. P_2 กับ P_4 ยาวเท่ากัน จึงใช้ลำดับมาถึงเป็น tie-break.

Process	Burst (ms)	CPU interval	First start S_i	Completion C_i
P_2	1	$[0, 1)$	0	1
P_4	1	$[1, 2)$	1	2
P_3	2	$[2, 4)$	2	4
P_5	5	$[4, 9)$	4	9
P_1	10	$[9, 19)$	9	19

ตัวอย่าง: เมื่อ P_2 และ P_4 จบที่เวลา 2, งานสั้นที่สุดที่เหลือคือ P_3; จึงจบที่ $2 + 2 = 4$. ทั้ง FIFO และ SJF ไม่หยุดกลาง burst ในโจทย์นี้; ตัวเลข Priority ไม่เกี่ยวกับสองนโยบายนี้.

เฉลยอัตนัยข้อ 1 (ต่อ): ไล่ Round Robin

เริ่มด้วย ready queue $[P_1, P_2, P_3, P_4, P_5]$. ในแต่ละรอบหยิบหัวคิวมาใช้ CPU ไม่เกิน 1 ms. ถ้าทำเสร็จให้ออกจากระบบ; ถ้ายังเหลือ burst ให้นำไปต่อท้ายคิว. ตัวอย่างรอบแรก: P_1 ใช้ 1 ms จาก 10 ms เหลือ 9 ms แล้วต่อท้าย; จากนั้น P_2 ใช้ 1 ms จบงาน.

เวลา (ms)	Process	Burst ที่เหลือ	ผลหลังจบ quantum
$[0, 1)$	P_1	9	กลับไปท้ายคิว
$[1, 2)$	P_2	0	เสร็จที่ $t = 2$
$[2, 3)$	P_3	1	กลับไปท้ายคิว
$[3, 4)$	P_4	0	เสร็จที่ $t = 4$
$[4, 5)$	P_5	4	กลับไปท้ายคิว
$[5, 6)$	P_1	8	กลับไปท้ายคิว
$[6, 7)$	P_3	0	เสร็จที่ $t = 7$
$[7, 8)$	P_5	3	กลับไปท้ายคิว
$[8, 9)$	P_1	7	กลับไปท้ายคิว
$[9, 10)$	P_5	2	กลับไปท้ายคิว
$[10, 11)$	P_1	6	กลับไปท้ายคิว
$[11, 12)$	P_5	1	กลับไปท้ายคิว
$[12, 13)$	P_1	5	กลับไปท้ายคิว
$[13, 14)$	P_5	0	เสร็จที่ $t = 14$
$[14, 15)$	P_1	4	เหลืองานเดียวในคิว
$[15, 16)$	P_1	3	ทำต่ออีกหนึ่ง quantum
$[16, 17)$	P_1	2	ทำต่ออีกหนึ่ง quantum
$[17, 18)$	P_1	1	ทำต่ออีกหนึ่ง quantum
$[18, 19)$	P_1	0	เสร็จที่ $t = 19$

เมื่อเหลือ P_1 เพียงงานเดียว CPU ยังแบ่งเวลาเป็น quantum ละ 1 ms; ใน Gantt chart จะรวมช่วงท้าย $[14, 19)$ เป็นกล่องเดียวก็ได้ แต่จุดแบ่งที่ 15, 16, 17 และ 18 ยังคงมีอยู่.

เฉลยอัตนัยข้อ 1 (ต่อ): คำนวณเวลาและค่าเฉลี่ย

นิยามที่ใช้คือ $T_i = C_i - A_i$ และ $R_i = S_i - A_i$, โดย S_i เป็นเวลาได้ CPU ครั้งแรก. ทุก process มาถึงเวลา $A_i = 0$, ดังนั้น turnaround เท่ากับ completion time และ response เท่ากับ first-start time.

Policy	Process	S_i	C_i	$T_i = C_i$	$R_i = S_i$
FIFO	P_1	0	10	10	0
	P_2	10	11	11	10
	P_3	11	13	13	11
	P_4	13	14	14	13
	P_5	14	19	19	14
SJF	P_1	9	19	19	9
	P_2	0	1	1	0
	P_3	2	4	4	2
	P_4	1	2	2	1
	P_5	4	9	9	4
RR	P_1	0	19	19	0
	P_2	1	2	2	1
	P_3	2	7	7	2
	P_4	3	4	4	3
	P_5	4	14	14	4

ตัวอย่างแทนค่าสูตร

FIFO: P_5 จบที่ 19 จึงได้ $T_5 = 19 - 0 = 19$ ms; เริ่มครั้งแรกที่ 14 จึงได้ $R_5 = 14 - 0 = 14$ ms.
RR: P_3 จบที่ 7 แม้ใช้ CPU รวม 2 ms จึงได้ $T_3 = 7 - 0 = 7$ ms; เริ่มครั้งแรกที่ 2 จึงได้ $R_3 = 2 - 0 = 2$ ms. เวลารอระหว่าง quantum หลังจากได้ CPU ครั้งแรกไม่ถูกนับซ้ำใน response time.

เฉลยอัตนัยข้อ 1 (ต่อ): ค่าเฉลี่ยและเกณฑ์ตรวจ

คำนวณค่าเฉลี่ย

- FIFO: $\overline{T} = (10 + 11 + 13 + 14 + 19)/5 = 67/5 = 13.4$ ms; $\overline{R} = (0 + 10 + 11 + 13 + 14)/5 = 48/5 = 9.6$ ms.
- SJF: $\overline{T} = (19 + 1 + 4 + 2 + 9)/5 = 35/5 = 7.0$ ms; $\overline{R} = (9 + 0 + 2 + 1 + 4)/5 = 16/5 = 3.2$ ms.
- RR: $\overline{T} = (19 + 2 + 7 + 4 + 14)/5 = 46/5 = 9.2$ ms; $\overline{R} = (0 + 1 + 2 + 3 + 4)/5 = 10/5 = 2.0$ ms.

สรุปข้อ (d): SJF ให้ average turnaround ต่ำสุดที่ 7.0 ms; RR ให้ average response ต่ำสุดที่ 2.0 ms. เป็นผลเฉพาะ burst และ arrival ชุดนี้ ไม่ได้แปลว่า scheduler เดิมจะชนะทุก workload.

เกณฑ์ตรวจ 8 คะแนน

- (a) 3 คะแนน: FIFO, SJF, RR อย่างละ 1 คะแนน; ดูลำดับงานและขอบเวลาบน Gantt chart.
- (b) 2 คะแนน: turnaround 15 ค่า; ให้ตามค่าที่คำนวณสม่ำเสมอจาก $C_i - A_i$.
- (c) 2 คะแนน: response 15 ค่า; ใช้เวลาได้ CPU ครั้งแรก $S_i - A_i$, ไม่ใช่เวลารอรวม.
- (d) 1 คะแนน: SJF สำหรับ minimum average turnaround (.5) และ RR สำหรับ minimum average response (.5).

หากคำนวณพลาดหนึ่งจุดแต่ใช้วิธีถูกในขั้นถัดไป ให้ follow-through credit และไม่หักความผิดเดิมซ้ำ.

เฉลยอัตนัยข้อ 2: MLFQ กับธนาคาร (4 คะแนน)

คำตอบตัวอย่างแบบเรียงความสั้น

> ช่วงลูกค้าแน่น ธนาคารอาจใช้ MLFQ โดยให้ลูกค้าใหม่เริ่มคิวความสำคัญสูง และเลือกบริการจากคิวสูงสุดที่ยังมีคนรอ หากอยู่คิวเดียวกันให้ผลัดกันแบบ RR; รายการที่ยังไม่เสร็จเมื่อใช้เวลาในระดับนั้นครบจึงเลื่อนไปคิวล่างซึ่งให้ช่วงบริการยาวขึ้น และควรเลื่อนผู้ที่รอนานกลับขึ้นคิวบนเป็นระยะเพื่อไม่ให้ถูกทิ้งไว้นาน การคัดแยกรายการเตรียมเอกสาร และเพิ่มช่องบริการตามภาระงานช่วยลดเวลาเฉลี่ยตั้งแต่เข้าคิวจนจบรายการ ทั้งนี้ควรพักธุรกรรมเฉพาะจุดปลอดภัย บันทึกความคืบหน้า และปรับนโยบายจากเวลาที่วัดได้จริง.

แยกกฎออกมาเพื่อให้เห็นว่าใช้ MLFQ อย่างไร

1. **เลือกคิวที่มี priority สูงสุดก่อน.** ถ้า Q0 มีลูกค้าพร้อมบริการ ให้เลือก Q0 ก่อน Q1 หรือ Q2; คิวล่างทำงานเมื่อไม่มีงานในคิวที่สูงกว่า.
2. **ใช้ RR ภายในคิวเดียวกัน.** ลูกค้าในระดับเดียวกันได้ช่วงบริการสั้น ๆ ตามลำดับ แล้ววนไปคนถัดไป จึงไม่ให้รายการเดียวครองช่องบริการตลอด.
3. **ให้รายการใหม่เริ่มคิวบนสุด.** ระบบยังไม่ทราบเวลาทำรายการจริงก่อนเริ่ม; รายการสั้นอาจจบจาก Q0 ส่วนรายการยาวจะใช้เวลาในคิวสูงต่อเนื่องมากกว่า.
4. **ลดระดับเมื่อใช้ allotment ครบ.** ถ้ารายการยังไม่จบหลังใช้เวลาสะสมครบโควตาของระดับนั้น จึงลดลงหนึ่งคิว; การ yield หรือพักก่อนหมด slice ไม่ควรล้างเวลาที่ใช้ไปแล้ว.
5. **ทำ priority boost เป็นระยะ.** เลื่อนงานที่ยังไม่เสร็จกลับขึ้นคิวสูงตามรอบ เพื่อไม่ให้งานซับซ้อนถูกงานสั้นแซงซ้ำจนแทบไม่ได้รับบริการ.

ตัวอย่างการเดินคิว (ตัวเลขสมมติ ไม่ใช่ค่าที่โจทย์กำหนด)

พิจารณาช่องบริการเดียว มี A, B, C มาพร้อมกันและใช้เวลา 1, 5, 3 นาทีตามลำดับ; Q0 ให้ slice 2 นาทีและ Q1 ให้ slice 4 นาที.

เวลา	คิว/ลูกค้า	เวลาที่เหลือ	ผล
0--1	Q0 / A	0 นาที	A เสร็จที่นาที 1
1--3	Q0 / B	3 นาที	ใช้ Q0 ครบ 2 นาที จึงย้ายไป Q1
3--5	Q0 / C	1 นาที	ใช้ Q0 ครบ 2 นาที จึงย้ายไป Q1
5--8	Q1 / B	0 นาที	B ใช้ต่อ 3 นาทีและเสร็จที่นาที 8
8--9	Q1 / C	0 นาที	C ใช้ต่อ 1 นาทีและเสร็จที่นาที 9

เมื่อทุกคนมาถึงเวลา 0, turnaround ของ A, B, C จึงเป็น 1, 8, 9 นาที. ตัวอย่างนี้แสดงว่าคิวสูงช่วยให้งานสั้นเสร็จเร็ว ขณะที่การลดระดับยังเปิดโอกาสให้งานยาวค่อย ๆ ทำต่อจนจบ.

ข้อเสนอปรับปรุงและข้อจำกัด

- ตรวจเอกสารและคัดประเภทบริการก่อนเข้าคิว เพื่อลดเวลาที่ช่องบริการหยุดรอข้อมูล; จัดผู้เชี่ยวชาญหรือเพิ่ม counter เฉพาะช่วงที่ข้อมูลชี้ว่าคุ้มกับกำลังคน.
- ตั้ง slice คิวสูงสั้นพอให้งานง่ายเริ่มเร็ว แต่ไม่สั้นจนเสียเวลาเปลี่ยนงานมาก; คิวต่ำให้ช่วงยาวขึ้นเพื่อลด overhead.
- บันทึกเวลา arrival, เริ่มบริการ, พัก, กลับมาทำต่อ และ completion แล้วคำนวณ turnaround จริงเพื่อปรับ slice, allotment และรอบ boost.
- ธุรกรรมธนาคารหยุดสลับไม่ได้ทุกจุดเหมือน CPU: ต้องพัก ณ checkpoint ที่ปลอดภัยและเก็บ progress; หลาย counter ทำให้บริการขนานได้แต่มีต้นทุนเพิ่ม.

MLFQ ไม่รับประกัน average turnaround ต่ำที่สุดทุกชุดงานเพราะไม่ทราบเวลาบริการล่วงหน้า; ควรประเมินทั้ง turnaround, ความเป็นธรรม และความถูกต้องของธุรกรรม.

เกณฑ์ตรวจ 4 คะแนน: ความเข้าใจและการเทียบกฎ MLFQ 2 คะแนน; policy ที่นำไปใช้ได้ 1 คะแนน; ข้อเสนอปรับปรุงพร้อมเหตุผลหรือข้อจำกัด 1 คะแนน. รับตัวอย่างอื่นที่อธิบายสอดคล้อง ไม่บังคับจำนวนคิวหรือเวลาเหมือนตัวอย่างนี้.

เฉลยอัตนัยข้อ 3: Page Replacement (8 คะแนน)

ตาราง FIFO, LRU และ OPT (6 คะแนน)

เริ่มด้วย 3 frames ว่าง; แต่ละคอลัมน์คือสถานะหลังรับ page reference นั้นแล้ว. H (hit) หมายถึง page อยู่ใน RAM อยู่ก่อน จึงไม่เปลี่ยน frame; F (fault) หมายถึง page ไม่อยู่ใน frame และต้องนำเข้ามา. เครื่องหมาย -- คือ frame ยังว่าง. ตารางย่อยจะไล่เหตุผลเฉพาะจังหวะที่ต้องเลือก victim.

FIFO: 13 faults, 7 hits

Ref. no.	1	2	3	4	5	6	7	8	9	10
Page	7	0	1	3	0	3	2	4	2	7
Frame 1	7	7	7	3	3	3	3	3	3	7
Frame 2	--	0	0	0	0	0	2	2	2	2
Frame 3	--	--	1	1	1	1	1	4	4	4
H/F	F	F	F	F	H	H	F	F	H	F

Ref. no.	11	12	13	14	15	16	17	18	19	20
Page	0	3	2	1	2	0	1	4	0	1
Frame 1	7	7	2	2	2	2	2	4	4	4
Frame 2	0	0	0	1	1	1	1	1	1	1
Frame 3	4	3	3	3	3	0	0	0	0	0
H/F	F	F	F	F	H	F	H	F	H	H

FIFO: ทำไมแต่ละ page จึงถูกนำออก Queue เรียงจาก page ที่เข้าก่อนสุดไปหลังสุด. References 1--3 เติม frame; เมื่อเกิด hit ลำดับ FIFO ไม่เปลี่ยน.

Reference no.	Page เข้า	Victim	Queue หลังแทนที่ (เก่า $\rightarrow$ ใหม่)
4	3	7	$[0, 1, 3]$
7	2	0	$[1, 3, 2]$
8	4	1	$[3, 2, 4]$
10	7	3	$[2, 4, 7]$
11	0	2	$[4, 7, 0]$
12	3	4	$[7, 0, 3]$
13	2	7	$[0, 3, 2]$
14	1	0	$[3, 2, 1]$
16	0	3	$[2, 1, 0]$
18	4	2	$[1, 0, 4]$

มี fault ตอนเติม frame 3 ครั้งแรกและ fault ที่ทำให้แทนที่อีก 10 ครั้ง รวม 13 faults; references ที่เหลือ 7 ครั้งเป็น hits.

เฉลยอัตนัยข้อ 3: LRU และเหตุผลเลือก victim

LRU (Least Recently Used) แทนที่ page ที่ไม่ได้ถูกใช้นานที่สุด ณ ตอนเกิด fault. ทุก hit ต้องปรับข้อมูลว่า page นั้นเพิ่งถูกใช้; ต่างจาก FIFO ตรงที่ hit เปลี่ยนลำดับความเก่า-ใหม่ได้.

LRU: 13 faults, 7 hits

Ref. no.	1	2	3	4	5	6	7	8	9	10
Page	7	0	1	3	0	3	2	4	2	7
Frame 1	7	7	7	3	3	3	3	3	3	7
Frame 2	--	0	0	0	0	0	0	4	4	4
Frame 3	--	--	1	1	1	1	2	2	2	2
H/F	F	F	F	F	H	H	F	F	H	F

Ref. no.	11	12	13	14	15	16	17	18	19	20
Page	0	3	2	1	2	0	1	4	0	1
Frame 1	7	7	2	2	2	2	2	4	4	4
Frame 2	0	0	0	1	1	1	1	1	1	1
Frame 3	2	3	3	3	3	0	0	0	0	0
H/F	F	F	F	F	H	F	H	F	H	H

ตรวจ victim ของ LRU ตัวเลขในคอลัมน์สุดท้ายคือ reference ครั้งล่าสุดที่ page นั้นถูกใช้ก่อนการแทนที่; เลือก page ที่เลขน้อยที่สุด.

Reference no.	Page เข้า	Victim	ครั้งล่าสุดของ victim
4	3	7	1
7	2	1	3
8	4	0	5
10	7	3	6
11	0	4	8
12	3	2	9
13	2	7	10
14	1	0	11
16	0	3	12
18	4	2	15

ตัวอย่างที่ reference 8: frames มี 3, 0, 2; ครั้งล่าสุดคือ 6, 5, 7 ตามลำดับ จึงแทนที่ 0 ซึ่งเก่าสุด. รวมกับ fault ตอนเติม frame ครั้งแรก 3 ครั้งจะได้ 13 faults และ 7 hits.

เฉลยอัตนัยข้อ 3: ตาราง OPT (ต่อ)

OPT แทนที่ page ที่จะถูกใช้อีกครั้งช้าที่สุดในอนาคต; ถ้าไม่ถูกใช้อีกเลย ให้ถือว่าไกลที่สุด. จึงต้องมอง reference string ที่เหลือไปข้างหน้า.

OPT: 10 faults, 10 hits

Ref. no.	1	2	3	4	5	6	7	8	9	10
Page	7	0	1	3	0	3	2	4	2	7
Frame 1	7	7	7	7	7	7	7	7	7	7
Frame 2	--	0	0	0	0	0	0	4	4	4
Frame 3	--	--	1	3	3	3	2	2	2	2
H/F	F	F	F	F	H	H	F	F	H	H

Ref. no.	11	12	13	14	15	16	17	18	19	20
Page	0	3	2	1	2	0	1	4	0	1
Frame 1	0	0	0	0	0	0	0	0	0	0
Frame 2	4	3	3	1	1	1	1	1	1	1
Frame 3	2	2	2	2	2	2	2	4	4	4
H/F	F	F	H	F	H	H	H	F	H	H

ไล่การตัดสินใจเมื่อ OPT เกิด fault

References 1--3 เติม frame ว่าง. เลขหลัง @ คือหมายเลข reference ที่ page นั้นจะปรากฏอีกครั้ง; "ไม่มี" หมายถึงไม่ปรากฏในส่วนที่เหลือ.

Ref. no.	Page เข้า	Frames ก่อน fault	การใช้ครั้งถัดไปของ resident pages	Victim
4	3	$7, 0, 1$	7@10, 0@5, 1@14	1
7	2	$7, 0, 3$	7@10, 0@11, 3@12	3
8	4	$7, 0, 2$	7@10, 0@11, 2@9	0
11	0	$7, 4, 2$	7 ไม่มี, 4@18, 2@13	7
12	3	$0, 4, 2$	0@16, 4@18, 2@13	4
14	1	$0, 3, 2$	0@16, 3 ไม่มี, 2@15	3
18	4	$0, 1, 2$	0@19, 1@20, 2 ไม่มี	2

ตัวอย่างที่ reference 4: จาก frames $7, 0, 1$ นั้น 7 จะกลับมาใช้ที่ 10, 0 ที่ 5 และ 1 ที่ 14; จึงนำ 1 ออกเพราะต้องรอนานที่สุด. OPT ได้ 3 faults ตอนเติม frame และอีก 7 faults ตามตาราง รวม 10 faults; ที่เหลือเป็น 10 hits. หากมี tie ระหว่าง page ที่ไม่ถูกใช้อีก เลือก victim ตัวใดก็ได้ที่ให้ผล optimal.

เฉลยอัตนัยข้อ 3 (ต่อ): EAT และเกณฑ์ตรวจ

ขั้นที่ 1: ทำหน่วยให้ตรงกัน

Memory access time $m = 10$ ns $= 0.00001$ ms. Disk access time $D = 10$ ms $= 10{,}000{,}000$ ns. จะใช้หน่วย ns ทั้งหมดหรือแปลงเป็น ms ทั้งหมดก็ได้ แต่ห้ามบวก ns กับ ms โดยตรง.

ขั้นที่ 2: หาโอกาส hit และ fault

มี references ทั้งหมด 20 ครั้ง. ให้ $p =$ faults$/20$ เป็นโอกาสเกิด page fault และ $h =$ hits$/20 = 1 - p$ เป็นโอกาส hit. ใช้โมเดล Lecture 6:

$$\text{EAT} = hm + pD = (1-p)m + pD.$$

ในที่นี้ D แทน disk access time ตามที่โจทย์ให้; ไม่เพิ่มต้นทุนอื่นที่โจทย์ไม่ได้ระบุ.

ขั้นที่ 3: แทนค่าและคำนวณ

- **FIFO:** $p = 13/20 = 0.65$ และ $h = 7/20 = 0.35$.
  $\text{EAT} = 0.35(10\,\text{ns}) + 0.65(10{,}000{,}000\,\text{ns})$
  $= 3.5 + 6{,}500{,}000 = 6{,}500{,}003.5$ ns $= 6.5000035$ ms ≈ 6.5 ms.

- **LRU:** fault/hit counts เท่ากับ FIFO สำหรับ string นี้, จึง $p = 0.65$, $h = 0.35$ และ EAT $= 6{,}500{,}003.5$ ns $= 6.5000035$ ms ≈ 6.5 ms.

- **OPT:** $p = 10/20 = 0.5$ และ $h = 10/20 = 0.5$.
  $\text{EAT} = 0.5(10\,\text{ns}) + 0.5(10{,}000{,}000\,\text{ns})$
  $= 5 + 5{,}000{,}000 = 5{,}000{,}005$ ns $= 5.000005$ ms ≈ 5.0 ms.

สรุปผล

Policy	Faults	Hits	p	EAT
FIFO	13	7	0.65	6.5000035 ms
LRU	13	7	0.65	6.5000035 ms
OPT	10	10	0.50	5.0000050 ms

OPT ให้ EAT ต่ำสุดในชุดนี้เพราะมี page faults น้อยที่สุด; FIFO กับ LRU ได้จำนวน fault เท่ากันโดยบังเอิญจาก reference string นี้ ไม่ได้แปลว่าจะเท่ากันทุกชุด. ถ้ารายงานโดยปัดเลข ได้ประมาณ 6.5 ms สำหรับ FIFO/LRU และ 5.0 ms สำหรับ OPT.

Rubric รวม 8 คะแนน

- ตาราง frames รวม 6 คะแนน: แต่ละ policy 2 คะแนน ให้ตาม frame contents 20 คอลัมน์ คอลัมน์ละ .1; ไม่หักเพราะไม่มีแถว hit/fault หรือการรายงาน counts แยกต่างหาก

- EAT รวม 2 คะแนน: สูตร .5; แปลงหน่วย .5; EAT แต่ละ policy ค่าละ $1/3$ ไม่บังคับเขียนข้อสรุปเพิ่มเติมที่โจทย์ไม่ได้ถาม

ให้ follow-through credit สำหรับ EAT ที่ใช้จำนวน fault จากตารางของผู้สอบอย่างถูกวิธี แม้ตารางก่อนหน้าผิด โดยหักข้อผิดเดิมในตาราง/จำนวน faults; ต้องไม่ให้คะแนนเต็มหากแทนค่าคนละหน่วยในสมการเดียวกัน

แผนผังเนื้อหาและแหล่งอ้างอิงสำหรับตรวจทบทวน

ความครอบคลุมของปรนัย

Lecture	จำนวนปรนัย	หัวข้อ
1	3	Privilege, trap, isolation
2	14	Process, state, context switch, wait
3	8	FIFO/SJF/STCF/RR, MLFQ, metrics
4	3	C storage class, partition fragmentation
5	7	Segmentation, paging, address translation
6	6	Virtual memory, demand paging, replacement

แหล่งเนื้อหา (path เทียบกับ root ของ repository)

- lectures/CPE333_OS_Midterm_StudyNotes.pdf และ Lecture1--Lecture6: ใช้กำหนดขอบเขต ไม่รวม threads/locks/deadlock ที่อยู่นอกชุด Lecture 1--6
- ปรนัย 1 ข้ออิงข้อสอบ CPE326 ปี 2561 (E03, ข้อ 7) และ 40 ข้ออิงชุดคำถามปี 2022 (E09); ตรวจตัวเลือกที่ไหลต่อข้ามหน้าจากไฟล์เต็ม 16 หน้า และใช้ Lecture 1--6 ตัดสินเฉลยเมื่อ annotation ขัดกับเนื้อหา
- quiz-and-exam/midterm/Backroom.pdf และข้อสอบปี 2558/2561: เทียบหัวข้อและแนวคำถาม ไม่คัดลอกหน้าที่มีคำตอบเขียนกำกับ; เมื่อเครื่องหมายเฉลยขัดกับ Lecture ให้ยึดแนวคิดใน Lecture (เช่น E09 Q6 และ Q24)
- quiz-and-exam/midterm/ยำรวมมิตรข้อเขียน.pdf หน้า 4--6 และ 10: Scheduling, bank analogy และ MLFQ rules; ตัวเลข burst ใช้ 10,1,2,1,5 รวม 19 ms ตามโจทย์ชุดนี้ ไม่ใช่ Quiz อีกชุดที่ $P_4 = 2$
- E09: os midterm exam.pdf หน้า 1--16 เป็น bank ปรนัยฉบับเต็ม; ตรวจภาพครบทุกหน้าและตามตัวเลือกที่ไหลข้ามหน้าใน Q4, Q7, Q10, Q17, Q20, Q27, Q30, Q40, Q43 และ Q50. E05 เป็นไฟล์สรุป quiz ปี 2022 ซึ่งมีเฉลยถึง Q40 แต่ไม่มีชุดตัวเลือกครบ
- E09 Q15 ใช้โดยระบุให้ชัดว่าเรียงตามลำดับการทำงานจนเสร็จ จึงตอบ FIFO; ตัวเลือก ``Both FIFO and RR'' คงตามต้นฉบับ แต่ไม่ได้ใช้เป็นเฉลยของ stem ที่ปรับให้แคบลง. Q45 ซ้ำกับ Q44 และตัดข้อที่อยู่นอกขอบเขตหรือยังมีคำตอบกำกวมออก
- quiz-and-exam/midterm/Quiz_BeforeMidterm.pdf: เทียบรูปแบบโจทย์ Scheduling และ fit policy

แม่แบบการพิมพ์

Palapluem/cpe222-past-exam-main, commit e87b379d09fdeb54f964d968c9599efdea68ba5d; main wrapper, cover และ chapter templates

คงการประกอบ PDF, ฟอนต์ THSarabunNew, margin, student header และกล่องหน้าปก; บรรทัดดอกจันบนปกใช้ข้อความตามแม่แบบที่ผู้ใช้ขอ ไม่ใช่การตรวจยืนยันการรับรองจริง

วันและเวลาตามข้อมูลที่ผู้ใช้ให้: 26 ตุลาคม 2569, 09.00--12.00 น. รายละเอียดคะแนนย่อยและเฉลยเป็นเกณฑ์ฝึกซ้อมแยกเล่ม แหล่งข้อสอบเก่าที่ใช้ระบุไว้ข้างต้น